

Frontend Engineer Take-Home Assignment

EJE Technologies — Next.js Website Build

Time Limit: 8–12 hours

Submission: GitHub repository link (public or private with access granted)

Deadline: 72 hours from receipt

Overview

You will build the marketing website for **EJE Technologies S.L.** — a Madrid-based company that builds efficiency-driven systems across physical and digital domains.

Use this as a reference for visual direction and structural quality: palantir.com

The goal of this assignment is **not** to see if you can finish quickly. It is to evaluate how you think about architecture, stability, data flow, state management, and design precision under real conditions. This assignment has intentional complexity — how you handle that complexity is what we are evaluating.

What You Are Building

Brand Identity

- **Company:** EJE Technologies S.L.
 - **Tagline:** *Efficiency, Enforced.*
 - **Tone:** Institutional, precise, technical — not startup-casual. Think defense contractor meets modern SaaS.
-

Page Structure

Hero Section

- **Headline:** We design systems that eliminate structural waste.
- **Subheading:** Eje Technologies builds and operates efficiency-driven solutions across physical and digital domains — reducing wasted water, energy, time, and human attention through enforceable systems.
- Two primary CTA buttons: **Explore Dancing Drops** / **Explore AI Code Reviewer**

Section 1 — The Thesis

Present the following as structured, readable content (not a bullet dump):

Fragmented systems waste resources. Households run half-empty washing machines. Software teams generate code faster than they can review it. Urban infrastructure and human cognition are under-optimized.

We build enforcement layers. We formalize rules. We embed them into workflows. We measure the deltas.

Efficiency is not a promise. It is a system outcome.

Section 2 — Our Products

Dancing Drops

- Urban laundry optimization, powered by batching and operational discipline
- Pickup & delivery in Madrid
- Professional load optimization
- Reduced water and energy per garment
- Measurable service reliability
- *Outcome: Time recovered. Resources conserved. Infrastructure utilized efficiently.*

- Link: [Learn more →](#)

AI Code Reviewer

- Project-aware architectural enforcement for software teams
- Pull-request level review
- Project-specific rule ingestion
- Context-aware feedback
- Cloud-first deployment
- *Outcome: Reduced review load. Lower technical debt. Enforced architectural consistency.*
- Link: [Learn more →](#)

Section 3 — Live Metrics Dashboard

This section replaces the static "Measurable Impact" section. It must be **dynamic and interactive** — see [Technical Requirements → Dynamic Metrics](#) for full implementation spec.

Section 4 — Governance & Discipline

Eje Technologies S.L. operates under clear IP ownership, GDPR-compliant data governance, defined operational thresholds, and financial discipline with transparent reporting.

Public funding, private customers, and operational partners require the same standard: measurable execution.

Section 5 — Built for Execution

Present these as declarative principles:

- We do not scale before optimizing.
- We do not automate before formalizing rules.
- We do not expand before validating margins.

Efficiency precedes growth.

Footer

- About Eje Technologies
 - Sustainability & Impact
 - Governance & Compliance
 - Dancing Drops
 - AI Code Reviewer
 - Contact
-

Technical Requirements

Stack (mandatory)

- **Next.js 14+** with App Router
- **TypeScript** — strict mode on, no any types
- **Tailwind CSS** for styling
- No component libraries (no shadcn, MUI, Chakra, etc.) — build from scratch

Required Directory Structure

Your project must follow this structure exactly. Deviating without documented reason in your README is a disqualifier.

```
/
├── app/
│   ├── layout.tsx          # Root layout with metadata
│   ├── page.tsx            # Homepage (composes sections only — no logic here)
│   ├── globals.css
│   ├── api/
│   │   ├── metrics/
│   │   │   └── route.ts    # API route for metrics data
│   │   └── products/
│   │       ├── dancing-drops/
│   │       │   └── page.tsx
│   │       └── ai-code-reviewer/
│   │           └── page.tsx
│   └── components/
│       ├── ui/              # Reusable primitives (Button, Badge, AnimatedCounter, etc.)
│       ├── sections/        # Page sections (Hero, Products, Metrics, Governance, etc.)
│       └── layout/          # Header, Footer, Nav
├── hooks/
│   └── useMetrics.ts         # Custom hook for metrics data fetching + polling
├── lib/
│   ├── utils.ts
│   └── metrics.ts           # Metrics data logic and types
├── types/
│   └── index.ts             # All shared TypeScript interfaces and types
├── public/
│   └── assets/
├── tailwind.config.ts
├── next.config.ts
└── README.md
```

Feature 1: Dynamic Metrics Dashboard

Section 3 must be a **live, auto-refreshing metrics dashboard** — not static text.

API Route (/api/metrics)

Build a Next.js API route that returns simulated operational metrics. The data should feel realistic, not random. Use seeded or time-based variation so numbers change gradually on each poll, not wildly on every request.

The response shape must match this TypeScript interface exactly:

```
typescript

interface MetricCategory {
  id: string;
  label: string;
  metrics: Metric[];
}

interface Metric {
  id: string;
  label: string;
  value: number;
  unit: string;
  trend: "up" | "down" | "stable";
  delta: number; // change since last reading, as a percentage
}

interface MetricsResponse {
  timestamp: string; // ISO 8601
  categories: MetricCategory[];
}
```

Populate with at least two categories (`laundry`) and (`software`) and at least 3 metrics each. Examples:

Category	Metric	Unit
Laundry	Machine Load Utilization	%
Laundry	Urban Trips Eliminated (this week)	trips
Laundry	Avg. Water Saved Per Cycle	liters
Software	PRs Reviewed (this month)	reviews
Software	Avg. Review Turnaround	hours
Software	Rework Cycles Prevented	cycles

Custom Hook (`useMetrics`)

Build a `useMetrics` custom hook that:

- Fetches from `/api/metrics` on mount
- Polls every **30 seconds** for fresh data
- Returns `{ data, isLoading, isError, lastUpdated }`
- Cleans up the polling interval on unmount (no memory leaks)
- Handles errors gracefully — the UI must not crash if the API fails

Dashboard UI

- Render each metric as a card with its label, value, unit, and trend indicator (arrow up/down/flat)
- Animate numbers when they change — build an `<AnimatedCounter />` component that smoothly counts from the old value to the new value (do not use a library for this — implement it with

`requestAnimationFrame` or a `useEffect` + `setInterval` approach)

- Show a subtle "Last updated: X seconds ago" indicator that updates in real time
 - Show a skeleton loading state on first load (build it — no library)
 - The dashboard must remain stable and not cause layout shift when new data arrives
-

Feature 2: Product Sub-pages with Shared Layout

The CTA buttons in the hero must link to two real pages:

- `/products/dancing-drops`
- `/products/ai-code-reviewer`

Each product page must:

- Share a common `ProductLayout` component (header with product name, back link, consistent spacing)
- Have its own unique content section expanding on the product details provided in the brief
- Use **Parallel Routes or Intercepting Routes** (Next.js App Router feature) OR justify in your README why you chose a simpler routing approach and what the tradeoffs are
- Include a mock "Request Access" form with the following fields: Name, Company, Email, Use Case (textarea). The form must:
 - Validate all fields client-side before submission (no library — write your own validation logic)
 - Show inline error messages per field (not an alert)
 - On submit, POST to a mock `/api/contact` route you build that returns `{ success: true }`
 - Show a success state after submission

- Be fully keyboard navigable and screen reader accessible
-

Feature 3: Theme System

Implement a **light/dark theme toggle** that:

- Defaults to dark mode (matching the Palantir-inspired aesthetic)
 - Persists preference to `localStorage`
 - Avoids flash of wrong theme on page load — solve this with a **script injected in `<head>` before `React hydrates`** (this is a known hydration challenge — how you solve it cleanly is part of the evaluation)
 - Uses CSS custom properties (not Tailwind's `dark:` class approach) for all color tokens — define a complete design token system in `globals.css`
-

Feature 4: Performance & SSR Correctness

- The Hero, Thesis, and Governance sections must be **Server Components** (no `"use client"`)
 - The Metrics Dashboard and the Theme Toggle must be **Client Components** — wrap them correctly with Suspense boundaries
 - Demonstrate correct use of `loading.tsx` or `Suspense` for at least one async data boundary
 - All `<Image>` components must have correct `width`, `height`, and `priority` flags — zero LCP penalty
 - Lighthouse Performance score ≥ 85 on desktop (screenshot this and include it in your README)
-

Stability Requirements

- **Zero hydration errors** in the browser console — this is a hard disqualifier
 - Zero missing `key` warnings in lists
 - No layout shift on load or when metrics refresh
 - Passes `next build` with zero errors or TypeScript errors
 - `eslint` passes with zero errors
 - All images use `next/image`, all links use `next/link`
-

Code Quality Requirements

- Components must be single-responsibility — nothing over ~100 lines without justification
 - No inline styles; all design via Tailwind utilities or CSS variables
 - Semantic HTML throughout: `<section>`, `<nav>`, `<article>`, `<header>`, `<footer>`, `<main>`
 - The contact form is fully accessible: labels associated with inputs, errors announced via `aria-live`, submit button has a descriptive `aria-label`
 - No `console.log` in production code
 - No `any` types — if you need an escape hatch, use `unknown` and narrow it
-

Design Requirements

- Dark background, precise typography, institutional aesthetic — reference Palantir's visual weight
- Full design token system in `globals.css` using CSS custom properties for colors, spacing, and type scale

- At least one scroll-triggered animation (CSS `IntersectionObserver` or Framer Motion — your choice, justify it in README)
 - Fully responsive: mobile (375px), tablet (768px), desktop (1440px)
 - The two product cards must be visually distinct but share the same design language
 - Typography must convey authority — avoid Inter, Roboto, or Arial; choose something with character and justify the choice
-

What We Are Evaluating

Area	Weight	What We Look For
Stability & Build	20%	Zero hydration errors, no console warnings, <code>next build</code> passes clean
Architecture & Structure	20%	Directory structure, Server vs Client component decisions, data flow clarity
Feature Completeness	20%	Metrics dashboard with polling, form with validation, theme toggle with no flash
Code Quality	20%	TypeScript precision, hook design, no shortcuts, readable and maintainable
Design & UX	20%	Institutional tone, responsive, <code>AnimatedCounter</code> , skeleton states, accessibility

Submission Requirements

Your GitHub repository must include:

1. **README.md** with:

- Setup and run instructions
- Your font and color system rationale
- Explanation of your Server vs Client component split and why
- How you solved the theme flash problem
- Your `AnimatedCounter` implementation approach
- Lighthouse performance screenshot (desktop)
- Tradeoffs made and what you'd improve with more time

2. A **working production build** — run `npm run build` before submitting. If it fails, do not submit until it passes.

3. No `.env` secrets committed. Use `.env.example` with placeholder values.

Explicit Disqualifiers

We will immediately reject submissions that:

- Have **any** hydration error in the browser console
- Use a component library (shadcn, MUI, Chakra, etc.)
- Use a form validation library (react-hook-form, zod, yup) — write your own validation
- Use a counter animation library — the `AnimatedCounter` must be hand-built
- Have `any` types in TypeScript
- Fail `npm run build`
- Have an empty or missing README
- Use a flat file structure

- Flash the wrong theme on page load and have no explanation for how they attempted to solve it
 - Leave the metrics dashboard as static, hardcoded content — polling is required
-

A Note on AI Usage

You may use AI tools. We expect you to. However, **you are responsible for every line of code in your repository**. During the follow-up interview, we will open your editor and ask you to:

- Explain your `useMetrics` hook line by line
- Walk us through how your theme flash prevention works
- Modify the `AnimatedCounter` component live in front of us
- Justify one Server/Client component boundary decision

If you cannot do these things, that tells us everything we need to know. Use AI as a tool, not as the engineer.

Questions?

If anything is unclear, email us before starting. Asking a precise clarifying question before beginning is a stronger signal than submitting something with wrong assumptions.

Good luck.

EJE Technologies S.L. — Efficiency, Enforced.